ian.ternational

Seu agente para vagas remotas

Um assistente para quem mora no Brasil e quer buscar vagas internacionais, entender as restrições de cada anúncio e decidir quais oportunidades valem a candidatura.

RELATÓRIO TÉCNICO / IAN TRINTA

O que você pode pedir

Pedido	Como o agente ajuda
“Encontre vagas remotas que aceitam Brasil.”	Busca anúncios, separa as vagas por elegibilidade e mostra o trecho que sustenta cada decisão.
“Qual delas combina com meu currículo?”	Recupera trechos do currículo e aponta habilidades em comum com o anúncio.
“Converta o salário da segunda para reais.”	Usa a lista da conversa para identificar a vaga e converte o salário quando o anúncio informa moeda e período.


Python


LangChain


OpenAI


FastAPI


Vue


LangSmith

Uma vaga remota pode excluir o Brasil

“Remote” não diz de onde você pode trabalhar. Se o anúncio omite a região, a vaga fica pendente. Se restringe a contratação aos EUA, o agente mostra por que ela saiu da lista.

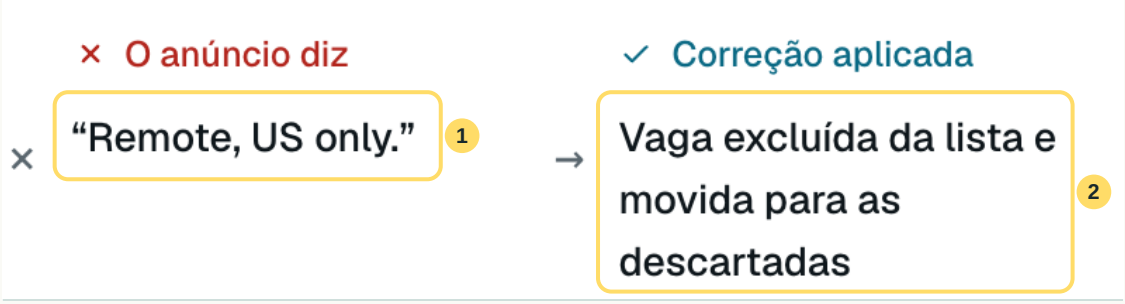


FIG. 01 Painel do app em modo demo. **1** O anúncio restringe a vaga aos EUA. **2** O agente mostra a decisão ao lado da fonte.

A chain transforma o currículo em dados

Para comparar o anúncio com o candidato, o sistema precisa de campos conhecidos, como habilidades e idioma. A primeira versão usa uma sequência fixa.

Prompt	→	Extração	→	Parser	→	Validação
Pede apenas dados explícitos.		Regras locais ou LLM.		Converte a saída JSON.		Confere o esquema do perfil.

Entrada do exemplo local: Data / Backend: Python, SQL, Pandas, PostgreSQL, pgvector, Milvus, MongoDB, REST APIs

Origem	Campo preenchido	Campo sem informação
Notebook 02. Currículo fictício.	`ingles = 'B2'`	`pretensao_salarial = None`
Currículo de Ian. Recorte executado localmente.	`Python, PostgreSQL`	`pretensao_salarial = null`

Por que LCEL? O mesmo fluxo aceita regras locais ou um LLM sem mudar o JSON. O recorte do autor usou regras locais. O notebook 02 demonstra também `@chain`, *invoke*, *batch* e *stream*.

O agente pode pedir uma ferramenta

Extrair um perfil é um caminho fixo. Buscar uma vaga exige decidir se vale consultar uma fonte e usar o resultado antes de responder.

ReAct clássico. Notebook 03	Executor do chat. Suíte real

Action: `buscar_vagas`
Action Input: `Python`
Observation: Remotive devolveu anúncios, entre eles Senior React Full-stack Developer.
	Rota: `busca`
	Chamadas estruturadas registradas:
	`buscar_vagas`
	`recuperar_contexto`

SAÍDA 02 À esquerda, callbacks de uma execução com OpenAI e consulta HTTP (687 ms na ferramenta); a observação está resumida. À direita, chamadas do modelo real com dados fixos.

O ciclo é parecido nos dois casos. O clássico transmite ação e argumento em texto e encerra com a resposta do próprio agente. O chat usa chamadas estruturadas dentro de um roteiro que ainda aplica regras e verifica a evidência.

Ação		Observação		Próximo passo
O modelo indica a ferramenta.	→	O app devolve o resultado.	→	O modelo pede outra ação ou encerra.

POR QUE LANGCHAIN

LangGraph daria controle explícito sobre um grafo de estados. CrewAI e AutoGen são opções para coordenar vários agentes. Neste projeto, um só executor cobre o domínio e permite reaproveitar as chains do MVP. O RouterChain escolhe a rota, e os callbacks registram ação, observação e uso do modelo.

O prompt PTCF define papel, tarefa, contexto e formato. Nos exemplos, “Remote” deixa a vaga pendente; “US only” leva ao descarte. A instrução manda checar região, restrições, aderência e evidência nessa ordem. O callback registra ações e observações.

“A segunda vaga” tem significado

O app guarda a ordem da lista por sessão. Assim, um pedido de câmbio pode recuperar o segundo ID sem repetir a busca.

Turno	Pedido	Saída registrada
1	Busque vagas remotas de Python que aceitam candidatos no Brasil.	1. Aurora Labs **2. Python Platform Engineer · Orbit Works**
2	Converta o salário da segunda vaga para reais.	**R$ 32.400,00 por mês** 1 EUR = R$ 6,00 na cotação fixa do teste.

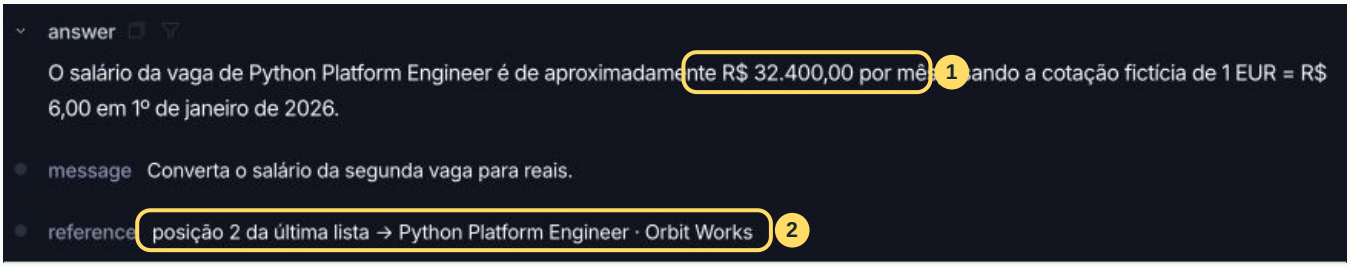


FIG. 02 Saída de *cambio-com-memoria* na suíte real. **1** Valor convertido. **2** A referência aponta para a Orbit, que ocupava a posição 2 na lista anterior.

Dois tipos de memória, dois usos

Na conversa. O notebook 04 demonstra `ConversationBufferMemory`. No agente final, `RunnableWithMessageHistory` lê e grava as mensagens da sessão. A posição da vaga vem da lista que o app guardou; janela, resumo e busca vetorial ajudam a limitar o contexto.

Entre conversas. No caso *preferencia-entre-sessoes*, o modelo chamou `guardar_memoria` ao receber o pedido para evitar Docker. O app recuperou a preferência em outra sessão do mesmo usuário e deixou a Orbit fora da lista.

ESCOLHA DE PROJETO

A memória curta acompanha a conversa atual. A longa guarda fatos, episódios e preferências de procedimento por usuário. A separação evita que uma referência como “a segunda” seja reaproveitada fora da sessão.

Cada fonte responde a uma pergunta

Um anúncio, uma cotação e um histórico de candidaturas exigem métodos diferentes. O agente reúne os resultados no mesmo turno quando o pedido precisa de mais de uma fonte.

Pergunta	Tipo e fonte	Decisão técnica
Quais vagas e quanto pagam?	APIs externas Remotive, Arbeitnow e AwesomeAPI	Busca em paralelo; o cálculo do câmbio segue uma regra fixa quando o anúncio informa moeda e período.
A quais vagas ainda não me candidatei?	Texto para SQL SQLite	O modelo gera SQL. O app permite leitura e restringe as candidaturas à sessão.
Qual vaga combina com o perfil?	Busca semântica Docling e FAISS	Docling extrai o texto do PDF. O FAISS consulta o índice existente e devolve os trechos mais próximos.

Uma consulta em português perdeu a vaga certa

Consulta	Dois primeiros resultados	Aurora
desenvolvedor backend sênior	Moon Studio · Atlas Systems	Fora
Senior Backend Engineer	Aurora Labs · Atlas Systems	**1º lugar**

SAÍDA 04 Caso *rag-pt-en*, com `paraphrase-multilingual-MiniLM-L12-v2` local. Mesmo índice, mesmo corpus fictício, mesmo k = 2. Mudou apenas a consulta.

POR QUE REESCREVER

Os anúncios estavam em inglês. A primeira consulta recuperou documentos, mas deixou de fora o alvo conhecido. A reescrita recuperou Aurora sem reconstruir o índice; um único caso não mede a melhora geral da busca.

O integrador decide, executa e verifica

A chain inicial seguia passos fixos. O agente final é híbrido: o código organiza o turno e o modelo escolhe ferramentas dentro desse fluxo.

Intenção
Identifica a rota e as restrições.

→

AgentExecutor
O modelo pede ferramentas.

→

Verificação
Regras e NLI confrontam a evidência.

→

Resposta
Redige e guarda o turno.

O app usa `create_openai_tools_agent` com `AgentExecutor`. Se o modelo omitir uma ferramenta exigida pela rota, o código pode chamá-la. A avaliação registra essa chamada à parte.

Atlas: a mesma decisão, duas evidências

Execução	O que foi verificado	O que chegou à lista
Demo com contradição deliberada	Applicants based in Brazil are eligible. **Veredito: contradita.**	Atlas excluída.
Modelo real com dados fixos	Applicants based in Brazil are not eligible. NLI: neutral. **Veredito: confirmada.**	Atlas: descartado.

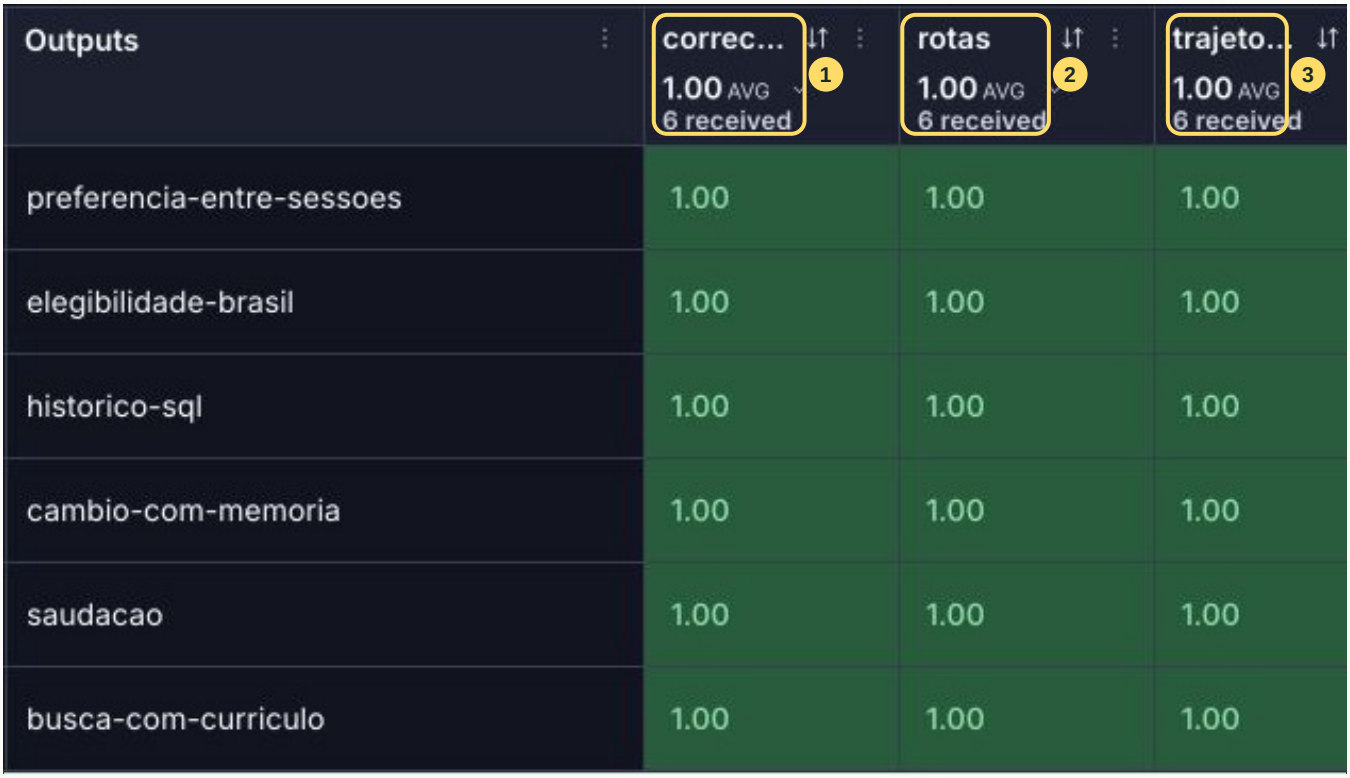
SAÍDA 05 O anúncio dizia “Remote, US only”. Na demo, a proposta dizia que o Brasil era elegível. No caso real, o agente fez a afirmação negativa: a regra geográfica e a checagem de coerência a sustentaram, enquanto o NLI ficou neutro.

Qual modelo faz o quê

Papel	Modelo e função
Interpretar, agir e redigir	`gpt-6-luna` · classifica a intenção, pede ferramentas, gera SQL e redige no modo real.
Recuperar trechos	`text-embedding-3-small` · transforma texto em vetores no modo real. A demo usa MiniLM multilíngue.
Verificar afirmações	`cross-encoder/nli-deberta-v3-small` · compara anúncio e afirmação antes da redação. No modo real, `gpt-6-luna` também checa coerência.

O que os testes mediram

A suíte principal avaliou gpt-6-luna em seis cenários. Anúncios, cotação e currículo ficaram fixos; a decisão do modelo e sua resposta foram geradas durante a execução.



Outputs	correc... 1.00 AVG 6 received (1)	rotas 1.00 AVG 6 received (2)	trajeto... 1.00 AVG 6 received (3)
preferencia-entre-sessoes	1.00	1.00	1.00
elegibilidade-brasil	1.00	1.00	1.00
historico-sql	1.00	1.00	1.00
cambio-com-memoria	1.00	1.00	1.00
saudacao	1.00	1.00	1.00
busca-com-curriculo	1.00	1.00	1.00

FIG. 03 LangSmith, suíte real. **1** Correção semântica. **2** Rotas. **3** Trajetória do modelo. Cada linha representa um caso.

Avaliação	Qualidade semântica	Ferramentas	Rotas
Demo com dados fixos	12/12	12/12 · execução	Não medida
Modelo real com dados fixos	6/6	6/6 · escolha do modelo	6/6

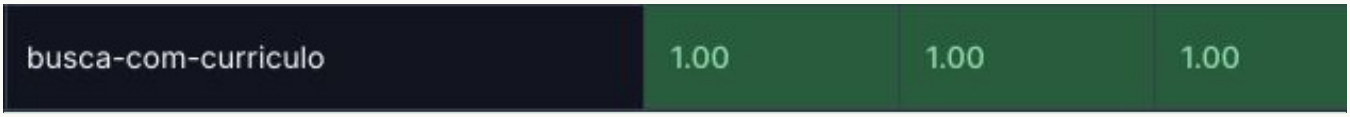
Correção semântica: na suíte real, `gpt-6-luna` atua também como juiz em uma chamada separada. Ele recebe o critério e o registro da execução. A demo também usa um juiz OpenAI, apesar de o agente ser roteirizado.

Rotas e ferramentas: o código compara os registros com o esperado em cada turno. Na suíte real, a ordem entre ferramentas independentes não importa; a trajetória só passa se o app não completar nenhuma chamada obrigatória.

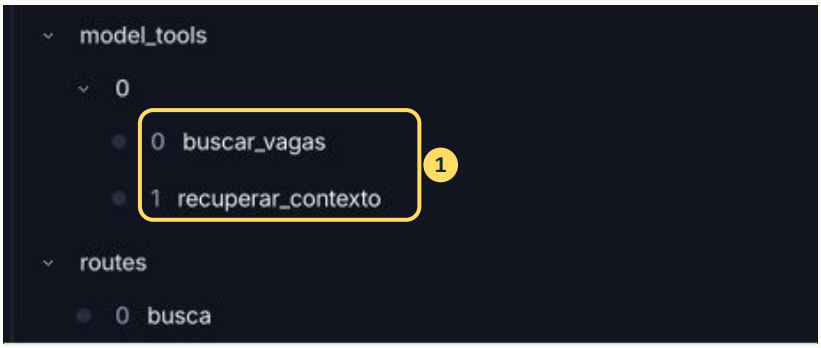
Abrir o experimento com modelo real

Por que este caso passou

O pedido foi buscar vagas para o Brasil e compará-las com o currículo fictício. O teste esperava que o modelo pedisse busca e recuperação documental no mesmo turno.



Esperado



Obtido

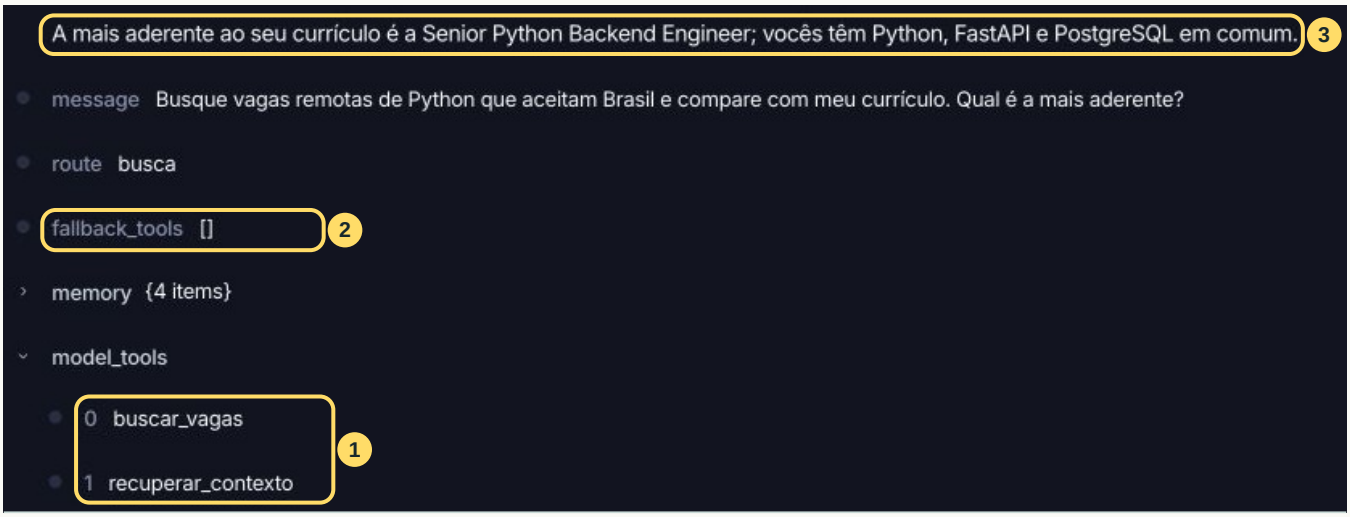


FIG. 04 Recortes da mesma execução: nota, referência e saída. **1** As duas ferramentas esperadas aparecem na saída. **2** O app não precisou completar chamadas. **3** Aurora lidera por Python, FastAPI e PostgreSQL em comum.

O acerto da consulta não garante a redação

As notas respondem a critérios definidos antes dos testes. Ler a saída completa mostra problemas que uma nota binária pode deixar passar.

O que a consulta fez	O que o usuário leu
SQLite retornou **Atlas Systems**. A consulta aplicou a faixa em USD e excluiu candidaturas enviadas da sessão.	Encontrei uma vaga, mas o rascunho não informa se ela paga acima de 5.000 USD por mês nem se você já se candidatou.

SAÍDA 06 Caso *historico-sql*, suíte real. O juiz aprovou o resultado esperado; a ressalva na resposta confunde o usuário.

Trecho da consulta	O que restringe
`currency = 'USD' AND salary_min > 5000`	Moeda e valor mínimo.
`NOT EXISTS (... status = 'enviada')`	Vagas sem candidatura enviada na sessão.

Uma melhoria é levar os filtros executados para o rascunho que o redator recebe e testar explicitamente se a resposta preserva esses filtros.

A consulta HTTP foi verificada à parte

O notebook 03 registra OpenAI chamando `consultar_apis` por function calling, com HTTP real. Em outro registro, com escolha roteirizada, Remotive (ok), Arbeitnow (ok) responderam, e a AwesomeAPI retornou a cotação USD de 2026-11-12. Nos seis casos do LangSmith, os dados externos ficaram fixos.

O QUE OS NÚMEROS NÃO MEDIRAM

As suítes têm casos e critérios diferentes; não medem ganho de desempenho entre modos. Os seis acertos descrevem uma execução com o mesmo modelo no papel de agente e de juiz. Uma revisão humana ajudaria a verificar as notas. Em uso real, fontes podem omitir dados, a redação pode variar e o histórico curto se perde ao reiniciar. O caso SQL também deveria testar anúncios sem período informado.

Como conferir o projeto

Os notebooks guardam as saídas de cada fase. A seção Evidência de cada notebook associa os itens da rubrica às células correspondentes.

Itens da rubrica	Onde encontrar a evidência
1.1 a 1.6 · Chain	Notebook 02 · arquitetura híbrida, planejamento e perfil em JSON. Compare LCEL e @chain.
2.1 a 2.6 · ReAct	Notebook 03 · escolha do orquestrador, ReAct clássico, RouterChain, HTTP, prompt PTCF e parser com recuperação.
3.1 a 3.6 · Memória	Notebook 04 · CoALA, comparação de ferramentas, ConversationBufferMemory, intenção e conversa em vários turnos.
4.1 a 4.5 · Ferramentas	Notebook 05 · APIs, SQL e recuperação. O Notebook 01 mostra como o índice foi preparado.
5.1 a 5.4 e 5.6 · Integração	Notebook 06 · verificação, executor e histórico. Resultados: suíte real e suíte demo.
5.5 · Streamlit	Não implementado. A interface do agente é Vue com FastAPI.

O README explica como rodar a demo sem credenciais. Para a defesa, o enunciado também prevê o uso da CLI: `.venv/bin/python -m scripts.cli`.

Para gerar este PDF com as dependências instaladas: `.venv/bin/python -m scripts.report`. O gerador usa as capturas e os resultados salvos; não consulta os serviços nem roda os testes de novo.

Fontes e uso de IA

Enunciado e rubrica · Artigo ReAct · Avaliação no LangSmith · Modelo NLI · Remotive · Arbeitnow · AwesomeAPI

OpenAI Codex auxiliou na implementação, nos testes e na redação. Claude Design auxiliou no protótipo e Claude Code na interface Vue e em suas rotas. As figuras de interface são capturas de execuções; as notas vêm dos experimentos identificados.